# AIOPS-03: Davis AI — Problems and Root Cause Analysis

> **Series:** AIOPS — Dynatrace Intelligence | **Notebook:** 3 of 8 | **Created:** May 2026 | **Last Updated:** 10/12/2026

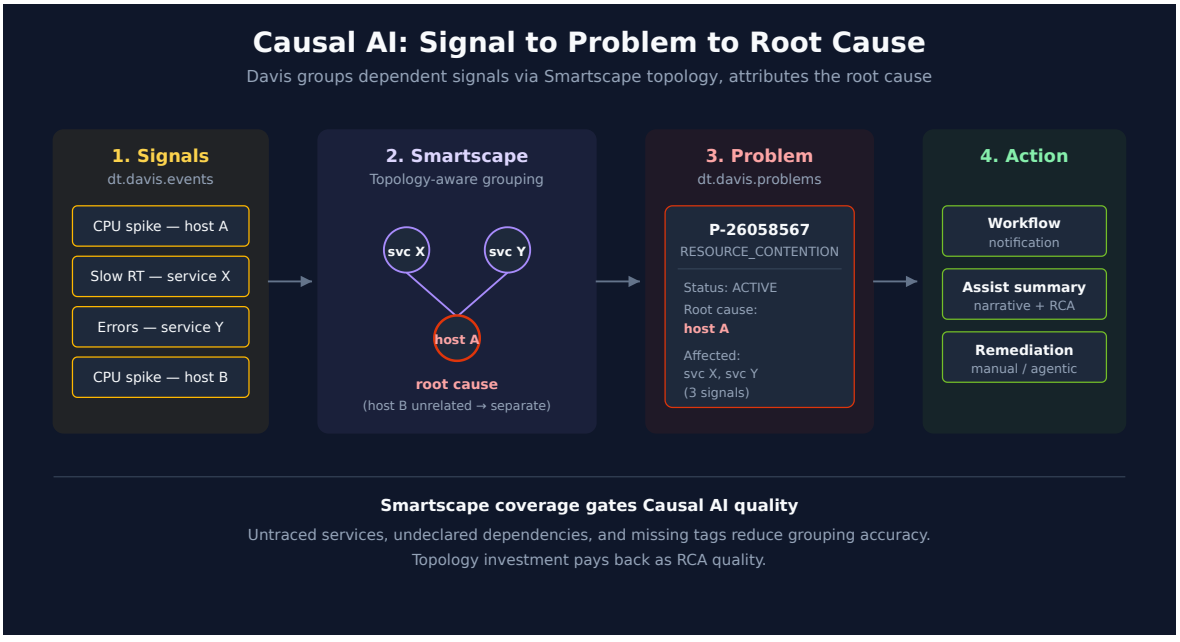
## Overview

Davis is the Causal AI engine. It watches the constant stream of anomaly signals (the raw `dt.davis.events`), groups related signals using Smartscape topology, and produces **problems** with a ranked root cause.

This notebook covers the Causal AI mechanism, the Problems app surface, the canonical DQL for querying problems, and the patterns for measuring detection quality.

**Audience:** SRE responding to incidents; platform admin tuning detection; observability lead reporting on MTTR.

**Outcome:** Working DQL for problem investigation, MTTR reporting, and root-cause analytics on your own data.



## Table of Contents


1. How Causal AI Groups Signals into Problems
2. Problem Lifecycle and Fields
3. The Two Data Objects: `dt.davis.problems` vs. `dt.davis.events`
4. Active Problem Feed
5. Problem Severity and Category Rollups
6. MTTR by Category and Service
7. Root Cause Entity Distribution
8. When No Root Cause Is Expected
9. Cross-Series Pointers


## Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS Gen3 with Grail |
| **Permissions** | `storage:events:read` |
| **Apps** | Problems app |
| **Topology** | Smartscape entities for the systems you care about (Causal AI's accuracy correlates with topology completeness) |

## 1. How Causal AI Groups Signals into Problems

Detectors fire constantly. Without grouping, every CPU spike, every slow request, every log error would page someone. The point of Causal AI is to ask: *which of these are the same incident?*

Davis answers using Smartscape — the dependency graph. Three signals on three different services that all depend on the same backing database go into one problem with

the database as root cause. Same three signals on unrelated services stay separate.

**This is why Smartscape coverage matters.** Causal AI's accuracy is bounded by topology completeness. Untraced services, undeclared dependencies, missing tags — all reduce the graph quality and produce worse grouping.

**Causal AI is deterministic, not statistical.** It walks the topology graph; it does not run an LLM or a black-box correlation. Two operators looking at the same data get the same root cause attribution.

## The correlation keys

Grouping is not a black box you have to take on faith — it runs on fields you can query.

**The universal rule is `dt.smartscape_source.id`.** Most Davis events carry this field — **84.9% on a validation tenant over 7 days, 08/11/2026** (186,914 of 220,206) — holding the Smartscape entity ID of whatever the signal is *about*. It is the rule that is universal, not the field's presence: the remaining share is exactly the population this section is about. Events naming the same entity within the correlation timeframe collapse into a single problem. The semantic dictionary types it `smartscapeId` at stability `stable` — it holds an entity ID, not a display name.

This is the field a custom alert most often gets wrong. The documentation asks for it to be set "to an existing Smartscape entity ID, like a host or service entity ID rather than an arbitrary string", the entity the same-entity rule above groups on. It does not say what happens to an event without one. The one documented fallback is narrower: an event posted through the Events API v2 without an `entitySelector` "is associated with the environment ( `dt.entity.environment` ) entity."

What the data shows is where to look: an event attached to the environment entity alone has nothing but the environment to be grouped on, and those events collect in large problems. On a validation tenant over 7 days to 10/12/2026, problems attached only to the environment averaged 14.7 events each, against 1.1 for every other problem. That is not the same thing as a missing source. Of the problem-eligible events with no `dt.smartscape_source.id`, 91% were still attached to a real entity, and 1,775 events from a cloud integration (all `Amazon EC2 CPU Utilization High`) carried a source yet were attached to the environment alone.

Treat environment-only attachment as the symptom to hunt — a problem with an implausibly broad, unrelated blast radius and no usable root cause — and a missing source in your own event templates as one cause of it. The remedy for your own detectors

is unchanged: set the event template's entity to a real host, service, or workload ID (AIOPS-02 §4); AIOPS-02 §8 has a query that finds these detectors.

**Topology rules merge across entity types.** Beyond exact-entity matching, Davis merges signals from entities in a known structural relationship — a process and the host it runs on are not two separate incidents:

| Grouping field | Entity types merged into one problem |
|---|---|
| `dt.smartscape.host` | `HOST`, `PROCESS`, `CONTAINER`, `DISK`, `NETWORK_INTERFACE` |
| `dt.smartscape.container` | `CONTAINER`, `PROCESS` |
| `dt.smartscape.process` | `SERVICE`, `PROCESS` |
| `dt.smartscape.k8s_pod` | `K8S_POD`, `CONTAINER` |
| `dt.smartscape.k8s_node` | `K8S_NODE`, `K8S_POD`, `CONTAINER` |
| `dt.smartscape.k8s_deployment` | `K8S_DEPLOYMENT`, `SERVICE` |
| `dt.smartscape.k8s_replicaset` | `K8S_REPLICASET`, `SERVICE` |
| `dt.smartscape.k8s_statefulset` | `K8S_STATEFULSET`, `SERVICE` |
| `dt.smartscape.k8s_daemonset` | `K8S_DAEMONSET`, `SERVICE` |
| `dt.smartscape.k8s_job` | `K8S_JOB`, `SERVICE` |

The table names the **grouping field**, not just a label, because that is what you filter on when you are working out why two alerts merged. Note the documentation introduces these as *"some examples of the grouping fields and potential entity types that can be merged"* — it is not an exhaustive list, so treat an unlisted merge as plausible rather than as a bug.

**Three further deduplication layers run on top:** time-based deduplication, so the same signal recurring inside the correlation window does not re-open a problem; causal merging across the dependency graph described above; and **frequent issue detection**, where Davis recognises a chronically recurring condition and stops raising it as a new problem on the reasoning that a permanent known issue is not news.

That last one is worth knowing before you conclude a detector has broken. A detector that "stopped alerting" may simply have had its condition classified as a frequent issue — check the event stream (`dt.davis.events`) rather than the problem feed to tell the two

apart. Frequent issue detection *"is being phased out on the latest Dynatrace platform"*; Dynatrace is instead improving the individual alert sources so frequent false positives are not generated. Use it to explain a quiet detector, not as a control to design around.

> **Sources:**
>
> - Avoid overalerting (DT docs) — all ten grouping fields, introduced as examples rather than an exhaustive list; *"make sure to set the dt.smartscape_source.id field to an existing Smartscape entity ID, like a host or service entity ID rather than an arbitrary string"*
> - Dynatrace Intelligence (DT docs)
> - Ingest an event — POST /api/v2/events/ingest (DT docs) — *"If not set, the event is associated with the environment"* (scoped to events posted through this API)
> - Transition from frequent issue detection (DT docs) — *"Frequent issue detection is being phased out on the latest Dynatrace platform."*
> - **Derived:** the "check `dt.davis.events` rather than the problem feed" diagnostic follows from frequent-issue suppression acting at the event-to-problem step, combined with the two-data-object split in §3.

## 2. Problem Lifecycle and Fields

**Status values:** `ACTIVE` and `CLOSED`. (Not `OPEN` / `RESOLVED` — common mistake.)

**Categories:** `ERROR`, `RESOURCE_CONTENTION`, `AVAILABILITY`, `SLOWDOWN`, `CUSTOM_ALERT`. Categories are stable; counts vary widely by environment.

**Severity** is a separate axis from category. The unified `event.severity` field is an integer 1–5 (ITIL-aligned) that propagates from the constituent alerts up to the parent problem — see the field table below. **It is typed `experimental` in the semantic dictionary** (verified 08/27/2026), while `event.category` and `event.kind` are `stable`: fine for triage and reporting, but do not hard-code an integration or an SLA contract against it without a fallback. Semantic Dictionary 1.348 (published 08/25/2026) promotes it to `stable` for Davis events and problems; a tenant checked 09/24/2026 still reported `experimental` — confirm on yours, and treat this caution as lifted once it reads `stable`.

**Key fields on a problem record:**

| Field | Description |
| --- | --- |
| `event.id` | Internal problem ID |
| `display_id` | Human-readable ID like `P-26058567` |
| `event.name` | Short problem title |
| `event.description` | Longer narrative |
| `event.category` | One of the categories above |
| `event.severity` | Unified severity as an integer 1–5 (1 = Critical, 2 = Major, 3 = Minor, 4 = Warning, 5 = Informational), aligned to the ITIL severity model. If an event source does not set it, the field is empty. Propagates from the constituent alerts up to the parent problem. Coexists with `event.category` — severity answers *how bad*, category answers *what kind*. |
| `event.status` | `ACTIVE` or `CLOSED` |
| `event.start` / `event.end` | Timestamps; `event.end` is null on active problems |
| `root_cause_entity_id` / `root_cause_entity_name` | Davis-attributed root cause |
| `affected_entity_ids` | Array of impacted entity IDs |
| `dt.davis.event_ids` | Array of constituent signal IDs from `dt.davis.events` |

Custom string-typed fields can be propagated onto problems via Settings — useful for team / alerting-profile / service-tier attribution. Numeric custom fields are not supported.

> **Changed (SaaS 1.346 — staged rollout from 08/25/2026): propagated problem fields keep only the latest value.** Verbatim: *"Propagated problem fields now store only the most recent event field value—they don't accumulate all historical values of an event field (for example, `trace_id` ), rather they save only the latest value."*
>
> This changes what a propagated field *means*, which matters more than it first appears. A field like `trace_id` on a long-running problem previously accumulated every value the underlying events carried; now it holds one — the most recent. Two consequences for anything you have built on those fields:
>
> - **Do not treat a propagated field as a complete history.** A query that pulled `trace_id` off a problem to enumerate every affected trace will, after this change,

return the last one only. Where you need the full set, go to the **events** (§3) rather than the problem record — the events are where the history actually lives, and that distinction is now load-bearing rather than stylistic.

- **Comparisons across the rollout window are not like-for-like.** A problem that opened before the change and persisted through it may show accumulated values; one opened after will not.

The upside is that a propagated field is now unambiguous — one value, the newest — rather than a list whose length depended on how long the problem stayed open.

**Sources:** What's new in Dynatrace SaaS 1.346 (DT docs) — the propagated-problem-field change quoted above. Standardized event severity (DT docs) — *"If an event source doesn't set severity, the field is empty."*

## 3. The Two Data Objects: `dt.davis.problems` vs. `dt.davis.events`

Sibling streams in Grail. **Use the right one.**

| Data object | Contains | event.kind | Use when you want |
|---|---|---|---|
| `dt.davis.problems` | Causal-AI-grouped problems | `DAVIS_PROBLEM` | The problem feed, MTTR, severity rollups |
| `dt.davis.events` | Raw, ungrouped signals | `DAVIS_EVENT` | Signal-level investigation, custom alert volume |

⚠️ Some docs and tutorials show `fetch dt.davis.events | filter event.kind == "DAVIS_PROBLEM"`. It returns zero rows — and the reason matters: this is a **filter on the wrong table, not a deprecated form**. `dt.davis.events` has only ever carried `DAVIS_EVENT`, so there is no older behavior to migrate from and nothing in your tenant to fix. Use `fetch dt.davis.problems` for problems. Verified 08/27/2026 (7 days): 206,023 rows in `dt.davis.events`, all `DAVIS_EVENT`; 0 when filtered to `DAVIS_PROBLEM`; control `dt.davis.problems` → 2,774.

## 4. Active Problem Feed

The most-used query in the series — what's broken right now, ranked by start time.

```
// Active problems right now — investigation feed
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| sort event.start desc
| fields display_id, event.name, event.category, event.start,
         root_cause_entity_name, affected_entity_ids
| limit 50
```

Filter by entity context to scope to a team or environment. Two common variants:

```
// Active problems in a specific Kubernetes namespace
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| filter k8s.namespace.name == "production"
| sort event.start desc
| fields display_id, event.name, event.category, root_cause_entity_name
| limit 50
```

```
// Active problems for a specific host group
// dt.host_group.id is NOT carried on problem records - resolve the group
through the affected hosts.
// Host groups are identified by NAME, not by a HOST_GROUP- entity ID.
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| expand host_id = dt.entity.host
| lookup [smartscapeNodes "HOST" | fields host_id = id_classic, host_group =
dt.host_group.id],
    sourceField:host_id, lookupField:host_id, fields:{host_group}
| filter host_group == "my-host-group"
| dedup event.id
| sort event.start desc
| fields display_id, event.name, event.category, root_cause_entity_name,
host_group
| limit 50
```

## 5. Problem Severity and Category Rollups

Two different axes, both useful in weekly operational reviews and for trending detection volume month over month:

- **Severity** ( `event.severity` , integer 1–5) — *how bad* — for prioritization, SLA reporting, and routing. 1 = Critical, 2 = Major, 3 = Minor, 4 = Warning, 5 = Informational (ITIL-aligned). `experimental` **stability** (until SD 1.348 reaches your tenant) — usable, but pin a fallback before routing on it. Expect a narrow spread unless your event sources set severity: on the validation tenant most days were 100% severity 3, and a 7-day window to 10/12/2026 held 3,557 severity-3 problems alongside 747 severity-1 and 788 severity-5 problems from a few sources that set their own. A one- or two-row rollup is a normal result, not a broken query.
- **Category** ( `event.category` ) — *what kind* — for spotting which failure modes dominate.

Start with the severity rollup, then break down by category.

> **Breaking change — SaaS 1.348 (pre-release; staged rollout planned from 09/22/2026):** *"Davis events and problems no longer default event.severity to 3."* Once it reaches your tenant, a problem with no severity set carries a null rather than 3 — the rollup below labels those *Not set* instead of letting them fall through to *Informational*, and a workflow filtering on `event.severity == 3` stops matching them. Until then, every unset severity still defaults to 3.
>
> **Sources:** What's new in Dynatrace SaaS 1.348 (DT docs), Semantic Dictionary changelog 1.348 (DT docs), Standardized event severity (DT docs) — *"Dynatrace aligns with the ITIL framework standard for IT management systems when classifying event and alert severity."*

```
// Last 7 days — problem count by severity (1=Critical … 5=Informational)
// Null is tested first: from SaaS 1.348 an unset severity is no longer
defaulted to 3.
fetch dt.davis.problems, from:-7d
| fieldsAdd severity_label = if(isNull(event.severity), "Not set",
    else: if(event.severity == 1, "Critical",
    else: if(event.severity == 2, "Major",
    else: if(event.severity == 3, "Minor",
    else: if(event.severity == 4, "Warning",
    else: "Informational")))))
| summarize problem_count = count(), by:{event.severity, severity_label}
| sort event.severity asc
```

```
// Last 7 days — problem count by category
fetch dt.davis.problems, from:-7d
| summarize problem_count = count(), by:{event.category}
| sort problem_count desc
```

```
// Last 30 days — daily problem volume
fetch dt.davis.problems, from:-30d
| makeTimeseries problems_per_day = count(), interval:24h, by:{event.category}
```

## 6. MTTR by Category and Service

Mean Time To Resolution — how long does Davis say a problem stayed `ACTIVE` before it transitioned to `CLOSED`. Compute it as `event.end - event.start` over closed problems.

Reporting principle: prefer **median** and **p95** over arithmetic mean. A single long-running availability problem (a host that was forgotten on the network) skews the average wildly.

```
// MTTR by category over last 30 days
fetch dt.davis.problems, from:-30d
| filter event.status == "CLOSED"
| fieldsAdd duration_hours = (event.end - event.start) / 1h
| summarize {
    median_mttr_hours = percentile(duration_hours, 50),
    p95_mttr_hours    = percentile(duration_hours, 95),
    problem_count = count()
  },
  by:{event.category}
| sort problem_count desc
```

## 7. Root Cause Entity Distribution

Which entities are most often attributed as root cause? A heavy concentration on a small set of services usually means either (a) you have real recurring instability there, or (b) topology is incomplete and Davis keeps falling back to the same nodes.

```
// Top 20 root cause entities — last 7 days
fetch dt.davis.problems, from:-7d
| filter isNotNull(root_cause_entity_name)
| summarize problem_count = count(), by:{root_cause_entity_name,
root_cause_entity_id}
| sort problem_count desc
| limit 20
```

## 8. When No Root Cause Is Expected

`root_cause_entity_name` empty on a problem is not automatically a detection gap. §1 already establishes the mechanism: Causal AI is a deterministic graph walk over Smartscape topology — it explains one signal by finding another signal, on a *connected* entity, that a dependency edge plausibly links to it. When there's no second signal to reach, or no edge connecting the signals that did fire, the walk has nothing to traverse. An empty root cause is that walk terminating correctly, not failing.

In community troubleshooting experience, three conditions account for most "expected empty" cases:

- **Single-event problems.** The constituent-event count is one. With nothing else in the window to correlate against, there's no second point for a chain to reach — Davis still names the affected entity, but there is nothing to name as its cause.
- **No topology edge between co-occurring failures.** Multiple signals fired in the same window, but Smartscape has no dependency relationship between the entities they're on — common with custom entities, unmodeled network paths, or topology that hasn't finished discovering a newer service. Multiple events without a connecting edge still produce no chain, because the walk needs both a second event *and* a path to it.
- **Infrastructure-level, single-entity impact.** The failing entity and the affected entity are the same node — a workload stuck in a bad state with nothing upstream feeding it. The fault is the entity itself, not something that happened to it, so there's nothing else in the graph to name.

None of these three call for action. They're worth distinguishing from a fourth case that does:

- **A real topology gap.** Multiple related entities were affected and a dependency chain plausibly exists between them, but Smartscape's model doesn't capture the edge — an undeclared dependency, a missing trace-context propagation hop, an entity type Smartscape doesn't yet model relationships for. This is the case where the topology-completeness work §1 already calls out genuinely improves the next problem's RCA.

**A practical way to tell them apart in your own data:** a null-root-cause problem with a single affected entity sits in the "expected empty" population; a null-root-cause problem with *multiple* affected entities is the one worth auditing for a topology gap.

Setting this expectation with stakeholders matters as much as the mechanism itself — a target of 100% RCA coverage treats every empty field as a miss, when a meaningful share of them are Causal AI doing exactly what it's designed to do.

> **Sources:** Avoid overalerting (DT docs) — *"alerts from different entities are merged when the entities are causally connected in the Smartscape topology"*, Dynatrace Intelligence (DT docs) — *"deterministic, causation-based analysis"*. **Derived:** an empty root cause is expected when no second signal sits on a causally connected entity.

**Track this as a trend, not a snapshot.** The bucketing query above characterizes a point in time; the same `root_cause_entity_id` field trended daily shows whether the expected-empty population is holding steady or whether something upstream — topology, tagging, correlation — is regressing.

```
// Characterize your own null-root-cause problems: single-entity (expected) vs multi-entity (audit for topology gap)
fetch dt.davis.problems, from:-30d
| filter isNull(root_cause_entity_name)
| fieldsAdd affected_count = arraySize(affected_entity_ids)
| fieldsAdd bucket = if(affected_count &lt;= 1, "single-entity (expected)", else: "multi-entity (audit for topology gap)")
| summarize problem_count = count(), by:{bucket}
| sort problem_count desc
```

```
// RCA attachment rate — % of non-duplicate problems with a populated root cause, trended daily
fetch dt.davis.problems, from:-30d
| filter not(dt.davis.is_duplicate)
| fieldsAdd rca_flag = if(isNotNull(root_cause_entity_id), 100.0, else: 0.0)
| makeTimeseries rca_attachment_pct = avg(rca_flag), interval:24h
```

## 9. Cross-Series Pointers

- **WFLOW-04** — wire active problems into notification workflows
- **DASH-03** — executive dashboards; **DASH-05** — engineering dashboards
- **AIOPS-04** — Dynatrace Assist generates problem-summary narratives via Generative AI
- **AIOPS-06** — Workflow tutorial: Summarize open problems with AI

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*